

scripts/autonomous-qa/lib-backend.sh — User Guide

Last verified: 2026-07-26 (LVA-018 script-docs backfill cycle)
Inheritance: HelixConstitution §11.4.18 (script docs); Lava §6.B ("Up" is not healthy), §6.H (credentials), §6.AK (device-gate auth)
Classification: project-specific

Overview

Sourceable library that brings **exactly ONE backend up at a time** (operator rule: never run both APIs concurrently). A lock file (scripts/autonomous-qa/.backend-active) enforces mutual exclusion — backend_up_* refuses when another backend holds the lock.

Two backend topologies:

- **Go backend (lava-api-go)** — podman compose on :8443, /health; the on-device client reaches it from the emulator via adb reverse (the topology-independent path for a containerized emulator).
- **Android backend (:api-app)** — the on-device embed (loopback); client + API both live on the emulator, no adb reverse needed.

Functions

Function	Purpose
backend_up_goapi	Bring the Go backend up via ./start.sh (lava-containers, api-go profile); poll https://127.0.0.1:8443/health up to 120 s; write the lock
backend_target_goapi <serial>	Set up adb reverse tcp:8443; print the URL the on-device client uses
backend_down_goapi	Stop via lava-containers (fallback ./stop.sh); clear the lock
backend_up_apiapp <serial>	Install the :api-app debug APK, drive the embed up on-device, poll its PUBLIC /health over real TLS; print (stdout) the URL the client uses

Function	Purpose
backend_target_apiapp <serial>	Print the on-device loopback URL (external-backend mode)
backend_down_apiapp [serial]	Force-stop + uninstall the api-app; clear the lock

Key details

- **./start.sh, never raw compose** for goapi: raw podman compose (podman-compose) cannot handle the network_mode: host services + the lava-net network; the Go orchestrator parses the compose correctly.
- **The api-app auto-start intent is the ONLY headless trigger** that binds the embed's listener: `am start -n digital.vasic.lava.api.dev/lava.api.app.MainActivity -ez lava.applink.START_API true` (AppLinkContract.EXTRA_START_API). A plain LAUNCHER launch leaves the engine Stopped. POST_NOTIFICATIONS is pre-granted so the start path never stalls on the runtime-permission dialog.
- **Readiness is a real TLS probe, not process liveness (\$6.B)**: the embed binds 0.0.0.0:8443 only after `ApiEngine.start()` succeeds, so a host-side `adb forward + /health 200` proves the server actually accepted a TLS connection and served. The on-device client itself uses plain loopback — no forward/reverse.
- Failure paths dump diagnostics (lava-containers status, podman logs lava-api-go, or filtered logcat) and return non-zero **without** writing the lock, so the matrix fails honestly.

Usage

```
source scripts/autonomous-qa/lib-backend.sh
backend_up_goapi                # or: API_URL="$
    (backend_up_apiapp "$SERIAL")"
API_URL="$(backend_target_goapi "$SERIAL")"
# ... run iterations ...
backend_down_goapi
```

Return: 0 = backend healthy (lock written); non-zero = failed (no lock).

Companion files

- `scripts/autonomous-qa/run-matrix.sh` — the consumer
- `start.sh` / `stop.sh` — the project's blessed Go-backend bring-up/teardown

- docs/ON_DEVICE_API.md — the :api-app embed architecture